SMART UNIVERSITY RESOURCE ALLOCATION SYSTEM

Iteration 44

Student Equipment Borrowing and Inventory Management

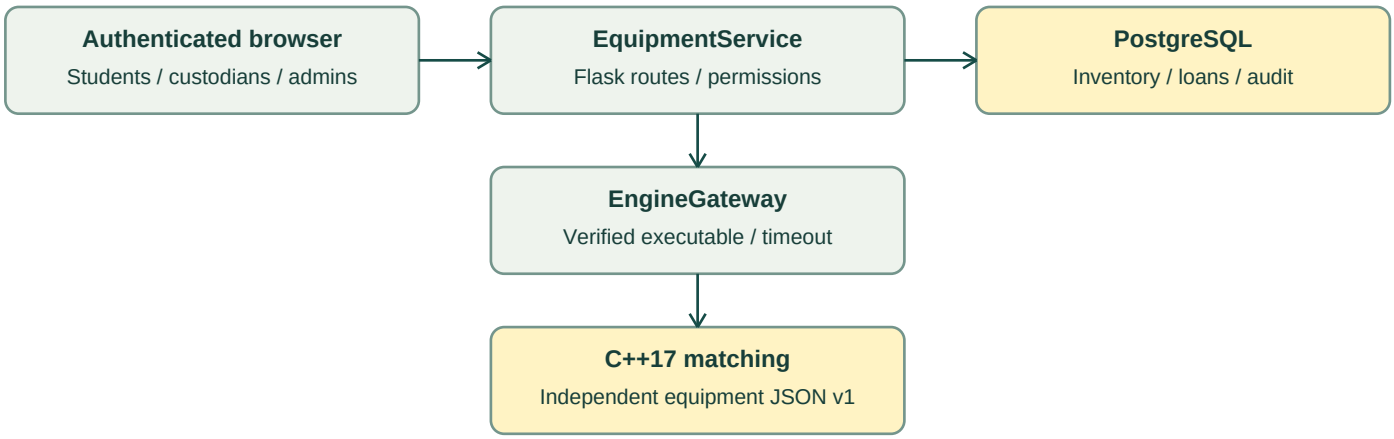
Najib Attar | UC-STU-03 | October 2026

1. Objective and delivered workflows

Deliver returnable university equipment borrowing while preserving authenticated appointments, student rooms, PostgreSQL authority and all five academic allocation strategies.

Students browse eligible types, specify quantities from one faculty, request a dated interval, check recommendations and submit for approval. The designated faculty custodian reserves exact units, verifies collection, records partial/full physical returns, handles condition and overdue issues, and decides extension requests. Administrators explicitly assign or replace custodians.

2. Architectural boundary



Room capacity and examination-sharing assumptions do not model returnable assets. A separate equipment type/unit inventory and JSON operation are used. C++ remains independent of PostgreSQL; no room strategy or appointment algorithm was rewritten.

3. Important components

Component	Responsibility
EquipmentMatchingService / EquipmentJsonProtocol	C++ eligibility, exact specification matching, distinct physical-unit selection and deterministic valid alternatives.
EquipmentService	Authenticated faculty-scoped workflow transactions, idempotency, approval, handover, returns, extensions and condition/history updates.
Equipment routes / Jinja / equipment.js	Role-specific forms and catalog faculty filtering within the existing shared authenticated layout; no browser allocation rules.
EngineGateway / NotificationService	Existing verified executable boundary and private event inbox reused; equipment events are published with lifecycle transactions.
equipment_demo / seed_equipment_demo.py	Reproducible fictional inventory; atomic seed refuses existing equipment configuration and creates no fake approved loans.

4. State transitions

Pending Approval -> Approved -> Borrowed -> Partially Returned -> Returned
Pending requests may be rejected. Pending/approved uncollected requests may be cancelled. Borrowed equipment cannot be cancelled as though it was never handed over. Return intent does not release a unit.

5. PostgreSQL additions

Nine additive tables; 47 application tables in total. Reviewed Alembic revisions 0044a-0044c preserve existing operational records. The full and focused ER diagrams are in docs/architecture/.

Table	Purpose
equipment_category	Extensible equipment classification.
equipment_type	Faculty ownership, category, specifications and configurable borrowing policy.
equipment_unit	Unique physical asset identifier, condition and condition notes.
equipment_custodian	One current authorized account for each faculty equipment desk.
borrow_request	Borrower, purpose, dated interval, state and idempotent form identity.
borrow_item	Requested type, quantity and requirements; composite same-faculty keys.
equipment_reservation	Exact unit, half-open reservation range, handover/return actors, timestamps and conditions.
borrow_extension	Requested later due date and explicit custodian decision.
equipment_event	Actor/time/faculty lifecycle and inventory audit; reused notification receipts.

6. Correctness, security and concurrency

Faculty-row locks serialize inventory mutations and the bounded matching call. Different faculties remain independent. GiST rejects overlapping unreleased unit reservations; composite foreign keys prevent mixed faculties/types. Deferred checks enforce full quantities and loan state; a physical-release check rejects release of unreturned handed-out assets.

Pending requests hold nothing. Due dates never imply return. Active unreturned units remain unavailable, including future matching; threatened existing reservations are flagged for staff. Missing/lost inspections do not claim physical return. Damaged/maintenance returns stay unallocatable. Extensions are never automatic.

Service-level ownership, active-account and current-custodian checks complement CSRF protection. Students see their own loan details only; public catalog data has no borrower identities or unapproved serial numbers. History and notifications commit together; engine failures roll the operation back.

7. Verification evidence

Executed check	Result
Fresh C++17 full suite	172 passed, 0 failed (160 existing + 12 equipment).
Python + real PostgreSQL full suite	287 run: 286 passed, 0 failed, 1 Windows-inapplicable POSIX check skipped.
Equipment-focused regression	28 tests: lifecycle, permissions, process race, rollback, migrations/drift, moderate/large demo.
CLI and browser acceptance	2 standalone CLI smokes; real browser request -> approval -> handover -> partial return -> full return and inbox verified.
Compatibility / integrity	All five strategy parity comparisons; migration and restore tests; syntax, schema documentation and workflow lint verified.

8. Design decisions and limits

Existing gateway/adaptation and service separation are reused; no artificial factory or room/exam strategy is added. Matching uses exact text specifications, stable inventory-ID ordering and explicit compatible alternatives, not machine learning. Faculty-scoped transactions favor correctness over high-throughput inventory processing. Overdue alerts refresh on inbox access; no background reminders or penalties. Active loans conservatively block new future allocations until verified return. No deployment or merge is performed as part of delivery.